

Introduction à R

Atelier intensif

Guillaume Boucher

INSPQ

Le 8 septembre 2026

Objectif

Cette formation a pour objectif de vous rendre autonomes dans votre apprentissage de R. Vous n'aurez pas à retenir toutes les fonctions ni toutes les syntaxes. L'important est surtout de comprendre les concepts fondamentaux, de savoir où chercher l'information et d'être capable de trouver et d'adapter des solutions à vos besoins.

Les bases

Commentaires

Le symbole pour les commentaires est le carré (#).

```
# Ci-dessous est une addition :  
4 + 3
```

```
#> [1] 7
```

```
# La ligne de code ci-dessous ne sera pas évaluée  
# 4 + 3
```

Arithmétique

```
# Addition
```

```
4 + 3
```

```
#> [1] 7
```

```
# Multiplication
```

```
4 * 3
```

```
#> [1] 12
```

```
# Exposant
```

```
4 ^ 3
```

```
#> [1] 64
```

```
# Soustraction
```

```
4 - 3
```

```
#> [1] 1
```

```
# Division
```

```
4 / 3
```

```
#> [1] 1.333333
```

```
# Modulo
```

```
4 %% 6
```

```
#> [1] 4
```

Variables

Une variable permet de stocker une valeur ou un objet pour ensuite l'utiliser ultérieurement. On utilise la flèche `<-` pour assigner à une variable une valeur. Le signe égal (`=`) fonctionne également, mais on priorisera ici la flèche.

```
x <- 4  
y <- 3
```

On peut ensuite utiliser les variables pour faire des opérations

```
x + y  # 4 + 3
```

```
#> [1] 7
```

```
z <- x - y  # 1 = 4 - 3  
z * y  # 1 * 3
```

```
#> [1] 3
```

Type de données

Les entiers (integer)

```
class(4)  # Un entier seul peut parfois être considéré numérique
```

```
#> [1] "numeric"
```

```
class(4L) # Ajouter L précise à R qu'on veut un entier
```

```
#> [1] "integer"
```

Exemples : année, âge, code d'identification numérique.

Numérique (numeric)

```
class(4)
```

```
#> [1] "numeric"
```

```
class(4.5)
```

```
#> [1] "numeric"
```

Exemples : taux, ratio

Nombres entiers en dehors de l'intervalle d'un entier standard seront aussi
numeric : $\pm 2\,147\,483\,647$

```
numeric2 <- 3e9 # équivalent à 3000000000
```

Exemple : identifiant des individus

Logique (logical)

```
class(TRUE)
```

```
#> [1] "logical"
```

```
class(FALSE)
```

```
#> [1] "logical"
```

Chaîne de caractères (character)

```
class("Bonjour")
```

```
#> [1] "character"
```

```
class('Bonjour')
```

```
#> [1] "character"
```

Une chaîne de caractères se fait avec des guillemets simples (' ') ou doubles (" "). On doit utiliser une barre oblique inversée (\) lorsque le même guillemet est utilisé à l'intérieur de la chaîne de caractères (voir exemples 2 et 3).

```
character1 <- "Il y a un atelier aujourd'hui"  
character2 <- 'Il y a un atelier aujourd\'hui'  
character3 <- "Voici des guillemets doubles : \" \""  
character4 <- 'Voici des guillemets doubles : " "'
```

Constantes

```
pi
```

```
#> [1] 3.141593
```

```
letters
```

```
#> [1] "a" "b" "c" "d" "e" "..."
```

```
LETTERS
```

```
#> [1] "A" "B" "C" "D" "E" "..."
```

```
month.abb
```

```
#> [1] "Jan" "Feb" "Mar" "Apr" "May" "..."
```

```
month.name
```

```
#> [1] "January" "February" "March" "April" "May" "..."
```

Fonctions

`nom_fonction(argument1, argument2, ...)`

```
sqrt(25)
```

```
#> [1] 5
```

```
round(pi, 2)
```

```
#> [1] 3.14
```

```
mean(1:5)
```

```
#> [1] 3
```

```
min(1:5)
```

```
#> [1] 1
```

```
max(1:5)
```

```
#> [1] 5
```

Vecteurs

Qu'est-ce qu'un vecteur ?

Liste d'éléments qui partagent tous exactement **le même type de données**.

On crée un vecteur avec la fonction `c()` : *combine values into a vector*.

```
num_vec <- c(1, 4.5, 10, NA)
chr_vec <- c("a", "b", "c", NA)
logi_vec <- c(TRUE, FALSE, FALSE, NA)
```

* Les valeurs manquantes sont désignées par NA que ce soit des nombres ou du texte.

Une valeur unique est aussi un vecteur

```
length(c(1, 2, 3))
```

```
#> [1] 3
```

```
length(c(1))
```

```
#> [1] 1
```

Puisqu'un vecteur contient un même type de données, il convertira les nombres en caractères au besoin.

```
c(1, 2, "bonjour")
```

```
#> [1] "1"      "2"      "bonjour"
```

```
class(c(1, 2, "bonjour"))
```

```
#> [1] "character"
```

Opérations sur un vecteur

À partir du vecteur `qte`

```
qte <- c(1, 3, 4, 5)
```

L'arithmétique se fait sur chaque valeur.

```
qte + 4
```

```
#> [1] 5 7 8 9
```

```
qte * 2
```

```
#> [1] 2 6 8 10
```

D'autres renvoient une seule valeur.

```
sum(qte)
```

```
#> [1] 13
```

```
mean(qte)
```

```
#> [1] 3.25
```

À partir des vecteurs qte1 et qte2

```
qte1 <- c(1, 3, 4, 5)
qte2 <- c(2, 1, 8, 4)
```

Les opérations se font selon la position des valeurs lorsque les vecteurs ont plusieurs valeurs et le **même nombre** de valeurs.

```
qte2 + qte1 # 2+1, 1+3, 8+4, 4+5
```

```
#> [1] 3 4 12 9
```

```
qte2 * qte1 # 2x1, 1x3, 8x4, 4x5
```

```
#> [1] 2 3 32 20
```

Sélection d'une valeur

Toujours à partir du vecteur `qte`

```
qte <- c(1, 3, 4, 5)
```

On sélectionne une valeur en utilisant les crochets `[]`.

Une seule valeur

```
qte[2]
```

```
#> [1] 3
```

```
qte[3]
```

```
#> [1] 4
```

Plusieurs valeurs

```
qte[c(2, 4)]
```

```
#> [1] 3 5
```

```
qte[c(2:4)] # ou qte[2:4]
```

```
#> [1] 3 4 5
```

```
qte[c(FALSE, TRUE, FALSE, TRUE)]
```

```
#> [1] 3 5
```

Comparaisons

Liste des opérateurs de comparaison :

- < : plus petit que
- <= : plus petit que ou égal à
- > : plus grand que
- >= : plus grand que ou égal à
- == : égal à
- != : différent, n'est pas égal à

À partir des vecteurs qte1 et qte2

```
qte1 <- c(1, 3, 4, 5)
```

```
qte2 <- c(2, 1, 4, 3)
```

On peut comparer les valeurs

```
qte2 > qte1
```

```
#> [1] TRUE FALSE FALSE FALSE
```

```
qte2 == qte1
```

```
#> [1] FALSE FALSE TRUE FALSE
```

Puisqu'on pouvait sélectionner des valeurs avec TRUE et FALSE

```
qte2[c(FALSE,TRUE,FALSE,TRUE)]
```

```
#> [1] 1 3
```

Et qu'une comparaison retourne un vecteur TRUE et FALSE

```
qte2 > qte1
```

```
#> [1] TRUE FALSE FALSE FALSE
```

On peut combiner les deux concepts

```
qte2
```

```
#> [1] 2 1 4 3
```

```
qte1
```

```
#> [1] 1 3 4 5
```

```
qte2[qte2 > qte1]
```

```
#> [1] 2
```

Lorsque les vecteurs n'ont pas le même nombre de valeurs, celui qui en a le moins doit être un multiple de celui qui en a le plus.

```
qte1 <- c(5, 1, 3, 4)
qte2 <- c(2, 4)
qte1 < qte2  # 5<2, 1<4, 3<2, 4<4
```

```
#> [1] FALSE TRUE FALSE FALSE
```

Sinon un message d'avertissement apparaît

```
qte1 <- c(5, 1, 3)
qte2 <- c(2, 4)
qte1 < qte2  # 5<2, 1<4, 3<2
```

```
#> Warning in qte1 < qte2: la taille d'un objet plus long n'est pas multiple
#> taille d'un objet plus court
```

```
#> [1] FALSE TRUE FALSE
```

Factor

Qu'est-ce qu'un *factor*?

Un factor en R est un type de données spécial à utiliser avec les chaînes de caractères.

```
sexe <- factor(c("F", "M", "M", "F", "F"))
```

```
#> [1] F M M F F  
#> Levels: F M
```

```
class(sexe)
```

```
#> [1] "factor"
```

Les Levels sont les valeurs qui existent à l'intérieur du factor (domaine de valeurs)

```
levels(sexe)
```

```
#> [1] "F" "M"
```

Visuellement c'est du texte, mais voici comment R traite un *factor* :

1. R stocke chaque catégorie sous forme de nombre entier (1, 2, 3...).
2. R associe chaque nombre à une étiquette de texte.
3. Cette astuce aide R à faire des calculs et des graphiques plus rapidement.

```
sexe <- factor(c("F", "M", "M", "F", "F"))
```

```
#>   VALEUR ETIQUETTE  
#> 1      1         F  
#> 2      2         M
```

```
factor(c("a", "b", "c"), levels = c("a", "b"))
```

```
#> [1] a      b      <NA>  
#> Levels: a b
```

Puisque "c" n'est pas une valeur de `levels`, le vecteur `c("a", "b", "c")` devient `c("a", "b", NA)`

Trier un *factor*

Par défaut les *Levels* sont les valeurs uniques en ordre alphabétique.

```
factor(c("petit", "grand", "moyen", "petit", "grand"))
```

```
#> [1] petit grand moyen petit grand  
#> Levels: grand moyen petit
```

Il est possible de donner un sens analytique aux données avec les arguments `levels` et `ordered`.

```
factor(  
  x = c("petit", "grand", "moyen", "petit", "grand"),  
  levels = c("petit", "moyen", "grand"),  
  ordered = TRUE  
)
```

```
#> [1] petit grand moyen petit grand  
#> Levels: petit < moyen < grand
```

Attention aux *factor*

Supposons le factor suivant :

```
presence <- factor(  
  x = c("Mercredi", "Lundi", "Samedi"),  
  levels = c("Dimanche", "Lundi", "Mardi", "Mercredi",  
             "Jeudi", "Vendredi", "Samedi"),  
  ordered = TRUE  
)  
sort(presence)
```

```
#> [1] Lundi    Mercredi Samedi
```

```
#> 7 Levels: Dimanche < Lundi < Mardi < Mercredi < Jeudi < ... < Samedi
```

R stocke chaque catégorie sous forme de nombre entier

```
as.integer(presence)
```

```
#> [1] 4 2 7
```

Dans le cas où on aurait des nombres sous la forme de factor

```
factor(c("200", "100", "200", "400"))
```

```
#> [1] 200 100 200 400
```

```
#> Levels: 100 200 400
```

Convertir en numeric ne retournera pas les valeurs attendues

```
as.numeric(factor(c("200", "100", "200", "400")))
```

```
#> [1] 2 1 2 3
```

Il faut d'abord convertir en character

```
as.numeric(as.character(factor(c("200", "100", "200", "400"))))
```

```
#> [1] 200 100 200 400
```

data.frame

Qu'est-ce qu'un data.frame

Un `data.frame` est une grille de données exactement comme une feuille Excel.

- Chaque ligne est une observation (un individu)
- Chaque colonne est une variable (son numéro d'identification, son âge, son nom...)

Chaque colonne est **un vecteur** et toutes les colonnes doivent avoir **la même longueur** ou être un multiple ou un facteur de toutes les autres (recyclage des données).

```
df <- data.frame( # 4 observations/lignes
  ID = 1:4, # 1 à 4
  SEXE = c("M", "F"), # 2 est facteur de 4, sera répété 2 fois
  AGE = c(45, 18, 25, 56),
  PROVINCE = "Québec", # 1 est facteur de 4
  TERRITOIRE = c(3, 16, 6, 14)
)
```

```
#>   ID SEXE AGE PROVINCE TERRITOIRE
#> 1  1    M  45   Québec          3
#> 2  2    F  18   Québec         16
#> 3  3    M  25   Québec          6
#> 4  4    F  56   Québec         14
```

```
class(df)
```

```
#> [1] "data.frame"
```

Fonctions de base

`View()` pour ouvrir dans l'éditeur

```
View(df)
```

`head()` et `tail()` pour voir les premières ou les dernières lignes d'une table.

```
head(df, n = 2)  # par défaut n=6 : head(df) = head(df, n = 6)
```

```
#>   ID SEXE AGE PROVINCE TERRITOIRE
#> 1   1    M  45  Québec          3
#> 2   2    F  18  Québec         16
```

```
tail(df, n = 2)
```

```
#>   ID SEXE AGE PROVINCE TERRITOIRE
#> 3   3    M  25  Québec          6
#> 4   4    F  56  Québec         14
```

str() pour visualiser la structure

```
str(df)
```

```
#> 'data.frame':    4 obs. of  5 variables:
#> $ ID          : int  1 2 3 4
#> $ SEXE        : chr  "M" "F" "M" "F"
#> $ AGE         : num  45 18 25 56
#> $ PROVINCE    : chr  "Québec" "Québec" "Québec" "Québec"
#> $ TERRITOIRE  : num  3 16 6 14
```

summary pour un résumé statistique rapide

```
summary(df)
```

```
#>      ID      SEXE      AGE      PROVINCE
#> Min.   :1.00  Length:4    Min.   :18.00  Length:4
#> 1st Qu.:1.75  Class :character 1st Qu.:23.25  Class :character
#> Median :2.50  Mode  :character Median :35.00  Mode  :character
#> Mean   :2.50
#> 3rd Qu.:3.25
#> Max.   :4.00
#>      TERRITOIRE
#> Min.   : 3.00
#> 1st Qu.: 5.25
#> Median :10.00
#> Mean   : 9.75
#> 3rd Qu.:14.50
#> Max.   :16.00
```

names() pour consulter ou renommer les colonnes

```
names(df)
```

```
#> [1] "ID"          "SEXE"        "AGE"         "PROVINCE"    "TERRITOIRE"
```

```
names(df) <- c("id", "sexe", "age", "prov", "terri")  
names(df)
```

```
#> [1] "id"      "sexe"    "age"     "prov"    "terri"
```

`dim()` pour voir le nombre de lignes et de colonnes

```
dim(df)
```

```
#> [1] 4 5
```

`nrow()` pour voir le nombre de ligne

```
nrow(df)
```

```
#> [1] 4
```

`ncol()` pour voir le nombre de colonnes

```
ncol(df)
```

```
#> [1] 5
```

Sélection des colonnes

Nom de la table + \$ + nom de la colonne

```
df$AGE    # vecteur
```

```
#> [1] 45 18 25 56
```

Nom de la table + [["nom de la colonne"]]

```
df[["AGE"]] # vecteur
```

```
#> [1] 45 18 25 56
```

Nom de la table + [, "nom de la colonne"]

```
df[, "AGE"] # vecteur
```

```
#> [1] 45 18 25 56
```

Nom de la table + ["nom de la colonne"]

```
df["AGE"] # data.frame
```

```
#>    AGE  
#> 1   45  
#> 2   18  
#> 3   25  
#> 4   56
```

Nom de la table + [, c("colonne1", "colonne2")]

```
df[, c("AGE", "SEXE")] # data.frame
```

```
#>    AGE  SEXE  
#> 1   45    M  
#> 2   18    F  
#> 3   25    M  
#> 4   56    F
```

Sélection des lignes

```
df[1, ] # première ligne
```

```
#>   ID SEXE AGE PROVINCE TERRITOIRE  
#> 1  1    M  45   Québec          3
```

```
df[2:4, ] # lignes 2 à 4
```

```
#>   ID SEXE AGE PROVINCE TERRITOIRE  
#> 2  2    F  18   Québec          16  
#> 3  3    M  25   Québec           6  
#> 4  4    F  56   Québec          14
```

```
df[c(2, 4), ] # lignes 2 et 4
```

```
#>   ID SEXE AGE PROVINCE TERRITOIRE  
#> 2  2    F  18   Québec          16  
#> 4  4    F  56   Québec          14
```

Filtrer

À partir d'un vecteur TRUE et FALSE

```
df$AGE > 30
```

```
#> [1] TRUE FALSE FALSE TRUE
```

On sélectionne les lignes qui respectent la condition

```
df[df$AGE > 30, ]
```

```
#>   ID SEXE AGE PROVINCE TERRITOIRE
#> 1  1  M  45  Québec          3
#> 4  4  F  56  Québec          14
```

```
df[df$TERRITOIRE %in% c(3, 6), ]
```

```
#>   ID SEXE AGE PROVINCE TERRITOIRE
#> 1  1    M  45   Québec          3
#> 3  3    M  25   Québec          6
```

%in% est utilisé pour sélectionner plusieurs valeurs, c'est équivalent à

```
df[df$TERRITOIRE == 3 | df$TERRITOIRE == 6, ]
```

```
#>   ID SEXE AGE PROVINCE TERRITOIRE
#> 1  1    M  45   Québec          3
#> 3  3    M  25   Québec          6
```

où la barre verticale (|) signifie ou/or.

```
df[df$TERRITOIRE %in% c(3, 6) & df$AGE > 30, ]
```

```
#>   ID SEXE AGE PROVINCE TERRITOIRE
#> 1  1    M  45   Québec          3
```

La perluète (&) signifie et/and.

Créer, modifier ou supprimer une colonne

Rappel pour créer ou modifier un vecteur :

```
nouveau_nom <- ancien_nom * 2 # Créer  
meme_nom <- meme_nom * 2 # Modifier
```

Puisque les colonnes d'un `data.frame` sont des vecteurs, la création ou la modification des colonnes se fait de la même manière :

```
nom_data$nouveau_nom <- nom_data$ancien_nom * 2 # Créer  
nom_data$meme_nom <- nom_data$meme_nom * 2 # Modifier
```

Pour supprimer une colonne :

```
nom_data$colonne_a_supprimer <- NULL
```

Créer une colonne

```
iris$sep_len_5plus <- iris$Sepal.Length >= 5
```

```
#>   Sepal.Length Sepal.Width ... sep_len_5plus
#> 1          5.1         3.5 ...          TRUE
#> 2          4.9         3.0 ...         FALSE
#> 3          4.7         3.2 ...         FALSE
#> 4          4.6         3.1 ...         FALSE
#> 5          5.0         3.6 ...          TRUE
#> 6          5.4         3.9 ...          TRUE
```

```
iris$Sepal.Length_x2 <- iris$Sepal.Length * 2
```

```
#>   Sepal.Length Sepal.Width ... Sepal.Length_x2
#> 1          5.1         3.5 ...          10.2
#> 2          4.9         3.0 ...           9.8
#> 3          4.7         3.2 ...           9.4
#> 4          4.6         3.1 ...           9.2
#> 5          5.0         3.6 ...          10.0
#> 6          5.4         3.9 ...          10.8
```

Modifier une colonne

```
iris$Sepal.Length <- iris$Sepal.Length * 2
```

```
#>   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
#> 1         10.2         3.5         1.4         0.2   setosa
#> 2          9.8         3.0         1.4         0.2   setosa
#> 3          9.4         3.2         1.3         0.2   setosa
#> 4          9.2         3.1         1.5         0.2   setosa
#> 5         10.0         3.6         1.4         0.2   setosa
#> 6         10.8         3.9         1.7         0.4   setosa
```

Supprimer une colonne

```
iris$Sepal.Length <- NULL
```

```
#>   Sepal.Width Petal.Length Petal.Width Species
#> 1         3.5         1.4         0.2   setosa
#> 2         3.0         1.4         0.2   setosa
#> 3         3.2         1.3         0.2   setosa
#> 4         3.1         1.5         0.2   setosa
#> 5         3.6         1.4         0.2   setosa
#> 6         3.9         1.7         0.4   setosa
```

Trier

La fonction `order()` indique les positions des valeurs dans le but de les réarranger.

```
order(iris$Sepal.Length)
```

```
#> [1] 14  9 39 43 42  4
```

```
order(iris$Sepal.Length, decreasing = TRUE)
```

```
#> [1] 132 118 119 123 136 106
```

```
iris[order(iris$Sepal.Length), ]
```

```
#>      Sepal.Length Sepal.Width Petal.Length Petal.Width Species
#> 14              4.3         3.0          1.1         0.1  setosa
#>  9              4.4         2.9          1.4         0.2  setosa
#> 39              4.4         3.0          1.3         0.2  setosa
#> 43              4.4         3.2          1.3         0.2  setosa
#> 42              4.5         2.3          1.3         0.3  setosa
#>  4              4.6         3.1          1.5         0.2  setosa
```

Listes

Qu'est-ce qu'une *list*?

Une liste permet de réunir une variété d'objets sous une même variable. Elle peut contenir tous les éléments qui existent, même une liste.

```
ma_liste <- list(  
  PI = pi,  
  LETTRES = LETTERS[1:5],  
  FLEURS = iris[1:3, ]  
)
```

```
#> $PI  
#> [1] 3.141593  
#>  
#> $LETTRES  
#> [1] "A" "B" "C" "D" "E"  
#>  
#> $FLEURS  
#>   Sepal.Length Sepal.Width Petal.Length Petal.Width Species  
#> 1           5.1           3.5           1.4           0.2  setosa  
#> 2           4.9           3.0           1.4           0.2  setosa  
#> 3           4.7           3.2           1.3           0.2  setosa
```

```
class(ma_liste)
```

```
#> [1] "list"
```

Sélectionner un élément d'une liste

```
ma_liste[2] # list
```

```
#> $LETTRES  
#> [1] "A" "B" "C" "D" "E"
```

```
ma_liste["LETTRES"] # list
```

```
#> $LETTRES  
#> [1] "A" "B" "C" "D" "E"
```

```
ma_liste[[2]] # vector
```

```
#> [1] "A" "B" "C" "D" "E"
```

```
ma_liste$LETTRES # vector
```

```
#> [1] "A" "B" "C" "D" "E"
```

Si l'élément d'une liste est un tableau

```
ma_liste$FLEURS
```

```
#>   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
#> 1          5.1          3.5          1.4          0.2  setosa
#> 2          4.9          3.0          1.4          0.2  setosa
#> 3          4.7          3.2          1.3          0.2  setosa
```

Les méthodes de sélection des colonnes sont les mêmes :

```
ma_liste$FLEURS$Sepal.Length
```

```
#> [1] 5.1 4.9 4.7
```

```
ma_liste[["FLEURS"]$Sepal.Length
```

```
#> [1] 5.1 4.9 4.7
```

```
ma_liste[["FLEURS"]][["Sepal.Length"]]
```

```
#> [1] 5.1 4.9 4.7
```

Packages

Qu'est-ce qu'un *package*?

Un *package* est un ensemble de ressources :

- Fonctions
- Données (data.frames)
- Documentation
- ...

On installe un package avec la fonction `install.packages()`.

```
install.packages("dplyr")
```

En date du 8 septembre 2026, il y a 24 894 *packages* de disponibles sur le **CRAN**.

Utiliser un *package*

Pour utiliser les ressources d'un package, on utilise la fonction `library()`.

```
library(dplyr)
```

La *library* est la bibliothèque et les *packages* sont des livres.

`library()` consiste à prendre un livre de la bibliothèque et le mettre sur son bureau afin de pouvoir l'utiliser.

En pratique, les gens utilisent souvent *package* et *librairy* comme synonymes. Le terme technique important à retenir est **package**. Une librairie est plutôt l'endroit où les *packages* sont installés.

Conclusion

- Il est important de savoir ce qu'est un `vector`, un `data.frame` et comment utiliser les fonctions.
- Il est impossible de tout connaître en R.
- Une recherche Google ou une question à Copilot peut rapidement répondre à votre question plutôt que de connaître les milliers de fonctions et objets disponibles dans le *package* R de base.